15.3
REST vs gRPC vs GraphQL

Choosing the right API protocol

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

Why Does API Protocol Matter?

When two services need to talk, they need to agree on HOW to talk. REST, gRPC, and GraphQL are three popular 'languages' services use. Picking the wrong one can slow your system down by 10x or make your mobile app waste data. This chapter explains when to use each.

> ■ **Simple Analogy:** REST is like sending letters (readable but slow). gRPC is like a phone call (fast, structured, but you need to know the number). GraphQL is like asking a waiter for exactly what you want from a menu.

REST — The Classic Web API

REST uses standard HTTP verbs (GET, POST, PUT, DELETE) on resource URLs. Data is usually JSON. It is simple, cacheable, and supported everywhere.

```
# REST — Getting a user
GET /api/v1/users/123
--> Response: { "id": 123, "name": "Alice", "email": "alice@x.com", "role":
"admin",
"address": {...}, "preferences": {...} }

# Problem: you only needed name + email but got EVERYTHING (over-fetching)
```

REST Characteristics

Characteristic	Detail
Protocol	HTTP/1.1 (text-based)
Data Format	JSON (human-readable)
Caching	Built-in via HTTP headers
Best For	Public APIs, browser clients
Used By	Stripe, Twitter, GitHub public APIs

gRPC — The Speed Champion

gRPC uses Protocol Buffers (binary format) over HTTP/2. You define your API in a .proto file and gRPC auto-generates client/server code in any language. It is 5-10x faster than REST for the same data.

```
// gRPC — Proto definition (contract-first)
service UserService {
rpc GetUser (GetUserRequest) returns (UserResponse);
rpc StreamUsers (Empty) returns (stream UserResponse); // streaming!
}
message GetUserRequest { string user_id = 1; }
message UserResponse { string id = 1; string name = 2; string email = 3; }

// Client code (auto-generated):
UserResponse resp =
stub.getUser(GetUserRequest.newBuilder().setUserId("123").build());
```

GraphQL — The Flexible Query Language

GraphQL lets the client ask for EXACTLY the data it needs — no more, no less. There is one endpoint (/graphql) and the client sends a query describing what it wants.

```
# GraphQL — Client asks for exactly what it needs
query {
user(id: "123") {
name # only name and recentOrders
recentOrders(last: 3) {
total
status
}
}
}
# Server returns ONLY name + 3 orders — nothing else
```

Comparison: REST vs gRPC vs GraphQL

Feature	REST	gRPC	GraphQL
Speed	Medium	FAST (binary)	Medium
Data Format	JSON	Protobuf	JSON
Fetching	Over/under-fetch	Exact fields	Exact fields
Streaming	No	YES	Subscriptions
Browser Support	YES	No (limited)	YES
Code Generation	No	YES (.proto)	Schemas
Best For	Public API	Internal SVCs	Flexible clients

■ **Real-World Example:** Netflix uses all three: REST for public developer APIs, gRPC for internal service calls (5-10x faster), and GraphQL Federation so mobile/TV/web devices request exactly the fields they need — reducing mobile bandwidth by ~30%.